FEATURE REPORT

Push to Clay

Sending the current filtered slice of the Companies dashboard to a Clay webhook — in-app, safe, and resumable.

Repo: moizali-devs/Scaletopia-Inventory · Epic #13 (closed) · Sub-issues #14–#19 (closed) · Date: 2026-07-01

1. The problem

Users can filter the Companies dashboard down to a slice (by niche, source, industry, country, employee size, search) but had **no way to send that slice to Clay** for outbound/enrichment without dropping to a standalone Supabase Edge Function script run by hand. There was no in-app trigger, no skip-already-sent tracking, and no success/failure feedback.

The goal was to reproduce the behavior of the original one-off script (`~/Downloads/index.ts`) *inside the app*, as a button next to Export.

2. The pivotal decision — we did *not* build the PRD's Edge Function

The PRD (`docs/prds/push-to-clay.md`) specified a fresh Supabase Edge Function (new Deno runtime, `supabase/` CLI project, `deno test`). Reading the actual codebase surfaced two findings that made that the wrong call:

> ### Finding 1 — the Edge Function couldn't own the thing that justified it
>
> The reference script filtered on `tags / client / quality_tier` . In this app the real company filters are `q, niche, source, industry, employee, country` — only `niche` overlaps. Worse, real filter resolution (`fetchFilteredRowsUncached`) does **in-app normalization** of country/industry/source (synonyms + casing) that a raw PostgREST query in Deno cannot reproduce. So filters *must* be resolved in Next regardless — leaving the Edge Function with nothing but an ID list to loop.

> ### Finding 2 — the repo already runs this exact shape in a Next route
>
> `app/api/import/stream/route.ts` already pushes ~20k rows to Supabase inside a Next route using `maxDuration = 300` , an SSE `ReadableStream` , and a server-only, Vitest-tested module. "Must use an Edge Function to dodge Vercel's timeout" was contradicted by working prior art.

We confirmed the pivot with the user and rebuilt the plan as a **Next route + SSE** feature: zero new toolchain, guaranteed filter fidelity, full pattern consistency. The one accepted trade-off: `CLAY_WEBHOOK_URL` lives as a server-only Next env var rather than a Supabase secret — equivalent in practice for a server-only app, and it still closes the original's real hole (accepting an arbitrary `webhook_url` from the client).

3. Architecture as built

```
components/companies/push-to-clay-button.tsx  (client)
  1. click  -> GET  preflight            -> { total_matched, eligible }
  2. radix AlertDialog shows real counts  (skips dialog if eligible === 0)
  3. confirm -> POST push route, read SSE -> live "Pushing X/Y..." on the button
```

```
  4. terminal "done" event                -> summary toast (pushed / errors / total)
        |
app/api/companies/push-to-clay/preflight/route.ts  (GET)
   -> resolveClayPushCounts(filters)

app/api/companies/push-to-clay/route.ts  (POST, force-dynamic, maxDuration=300)
   -> SSE stream -> runClayPush(filters, { onProgress })     <- the one tested seam
        - getAllFilteredCcompanies(filters)   (SAME path as CSV export = fidelity)
        - FRESH fetch of eligible rows WHERE pushed_to_clay IS NOT TRUE (uncached)
        - bounded-concurrency (8) webhook POST loop, +1 retry on transient status
        - incremental "mark pushed" in chunks -> idempotent & resumable

secret: process.env.CLAY_WEBHOOK_URL  (server-only, never NEXT_PUBLIC_)
```

4. Key engineering decisions (improvements over the original script)

Decision	Why it matters
Filter fidelity via the export path — push set comes from `getAllFilteredCompanies`, the identical code CSV export uses.	The pushed set is guaranteed to equal the on-screen set; no re-implemented filtering to drift.
Skip-already-pushed is fresh, never cached — pushed-status resolved at execution time, not via the 1-hour list cache.	A company pushed 2 minutes ago is correctly skipped; the cache would have re-shown it as unpushed for up to an hour.
Bounded concurrency (8) + 1 retry instead of the original's serial 50 ms loop.	~10× faster; realistic slices finish in seconds–tens of seconds, staying well inside the timeout.
Incremental marking in 200-row chunks as successes accumulate (vs one end-of-run update).	Makes the whole operation *idempotent & resumable*: a timeout or navigation is never data loss — re-running continues where it stopped. Satisfies "keep running for large sets" without a job queue.
Chunked `.in()` queries (200 ids) for fetch and mark.	Avoids the ~50 KB PostgREST URLs that the import module's header warns are slow to parse.
Preflight count endpoint feeds the confirm dialog.	Dialog shows the *real* eligible count, not the on-screen total — guards against an unintentionally broad push.
Failures isolated via `Promise.allSettled`; `failed_companies` capped at 20.	One bad webhook row never aborts the batch; toast stays readable.

5. Verification (what we actually ran)

New code

- **PASS** Typecheck — no errors in clay files
- **PASS** ESLint — no errors/warnings in clay files
- **PASS** `push-to-clay.test.ts` — **7/7** vs live Supabase

Covered: unpushed-only, partial-failure isolation, idempotency, empty set, missing config, 20-cap, preflight counts.

Non-regression

- **PASS** `custom-data.test.ts` — blocklist intact
- **OK** `pushed_to_clay(_at)` stay hidden from detail views
- **OK** No `supabase/` directory added

Feature touches no detail-view or blocklist code.

> **Not a blocker:** one pre-existing test — `getCompanies > search filters by name or domain substring` — fails on *clean HEAD too* (verified by stashing the feature). It's a data-dependent flaky search assertion, unrelated to this feature. The "2 failed" count is a duplicate from a stray worktree (see below), not two real failures.

6. What you need to do

#	Action	Why
1	**Set** `CLAY_WEBHOOK_URL` in `.env.local` and in the Vercel project env.	Without it the button errors "CLAY_WEBHOOK_URL is not configured". **Required before use.**
2	**Commit the feature.** It is currently uncommitted in the working tree (new files under `lib/clay/`, `app/api/companies/push-to-clay/`, the button, plus edits to `companies-results-client.tsx`, `.env.example`, `README.md`).	Nothing is committed yet; a review/PR is the natural next step.
3	**Prune the stray worktree**: `git worktree remove .claude/worktrees/agent-a3a89aaa85c40c8cc`.	It doubles every test run and inflates failure counts.
4	**Smoke-test in a real browser**: filter → Push → confirm → watch the count → summary toast; re-push to confirm skip.	The route/button paths are intentionally not unit-tested (matches Export precedent).
5	*Optional:* decide whether to add the import route's static-token guard to the push route.	Left off to match Export; flagged in #16 as a deliberate, reversible choice.
6	*Optional / separate:* fix the pre-existing flaky search test and the pre-existing lint errors in `people-results-client.tsx`.	Unrelated to this feature; worth a separate cleanup issue.

7. Deliverables

Path	Purpose
`lib/clay/push-to-clay.ts`	The tested seam: `runClayPush` + `resolveClayPushCounts`
`lib/clay/push-to-clay.test.ts`	7 Vitest cases against live Supabase

`app/api/companies/push-to-clay/route.ts`	POST — SSE push, `maxDuration=300`
`app/api/companies/push-to-clay/preflight/route.ts`	GET — confirm-dialog counts
`components/companies/push-to-clay-button.tsx`	Client button + radix AlertDialog + SSE reader
`components/companies/companies-results-client.tsx`	Wires the button beside Export
`.env.example`	Documents the server-only `CLAY_WEBHOOK_URL`

Generated for Scaletopia Inventory · Push to Clay feature · Epic #13. Filter fidelity, fresh skip-already-pushed, and incremental (resumable) marking are the three properties that make this correct under load.